

Python File Handling

Introduction

File Handling in Python refers to the process of creating, opening, reading, writing, appending, and closing files. It allows programs to store data permanently on a storage device instead of keeping it temporarily in memory.

File handling is essential for:

- Data Storage
- Log Management
- Report Generation
- Configuration Management
- Database Export/Import
- Cyber Security Log Analysis
- Data Science Applications

What is a File?

A file is a collection of data stored permanently on a storage device.

Examples:

student.txt

report.pdf

data.csv

image.jpg

log.txt

Why File Handling?

Without file handling:

```
name = "John"
```

- Data exists only while the program runs.
- After the program terminates: Data Lost
- Using files: Data Stored Permanently

Types of Files

1. Text Files

Store data in human-readable format.

Examples:

```
.txt  
.csv  
.html  
.py  
.json  
.xml
```

Example:

```
John  
25  
Python
```

2. Binary Files

Store data in binary format.

Examples:

```
.jpg  
.png  
.mp3  
.mp4  
.exe  
.pdf
```

File Handling Operations

Create File



Open File



Read / Write / Append



Close File

Opening a File

Python provides the open() function.

Syntax

```
file_object = open(filename, mode)
```

Parameters

Parameter	Description
filename	Name of file
mode	Access mode

File Modes

Mode	Description
r	Read
w	Write
a	Append
x	Create
rb	Read Binary
wb	Write Binary
r+	Read and Write
a+	Append and Read

Read Mode (r)

Used to read file contents.

Example

Assume:

```
student.txt
```

Contains:

```
John  
Python
```

Program:

```
file = open("student.txt", "r")

content = file.read()

print(content)

file.close()
```

Output

```
John
Python
```

Writing to a File (w)

Creates a new file or overwrites existing data.

Example

```
file = open("student.txt", "w")

file.write("Welcome to Python")

file.close()
```

File Content

```
Welcome to Python
```

Append Mode (a)

Adds new content to an existing file.

Example

```
file = open("student.txt", "a")

file.write("\nAdvanced Python")

file.close()
```

Output File

```
Welcome to Python
Advanced Python
```

Creating a New File (x)

Creates a new file.

```
file = open("newfile.txt", "x")
```

```
file.close()
```

If file already exists:

```
FileExistsError
```

Closing a File

Always close files after use.

```
file.close()
```

Benefits:

- Releases system resources
- Prevents data corruption
- Improves performance

Reading File Content

read()

Reads entire file.

```
file = open("student.txt", "r")
```

```
print(file.read())
```

```
file.close()
```

read(size)

Reads specified number of characters.

```
file = open("student.txt", "r")
```

```
print(file.read(5))
```

```
file.close()
```

Output

```
Welco
```

Reading Line by Line

readline()

Reads one line.

```
file = open("student.txt", "r")

print(file.readline())

file.close()
```

Output

```
Welcome to Python
```

Multiple readline()

```
file = open("student.txt", "r")

print(file.readline())
print(file.readline())

file.close()
```

Reading All Lines

readlines()

Returns list of lines.

```
file = open("student.txt", "r")

lines = file.readlines()

print(lines)

file.close()
```

Output

```
[
  'Welcome to Python\n',
  'Advanced Python'
]
```

Using Loops

```
file = open("student.txt", "r")

for line in file:
    print(line)

file.close()
```

Using with Statement

Best practice in Python.

Syntax

```
with open(filename, mode) as file:
    statements
```

Python automatically closes the file.

Example

```
with open("student.txt", "r") as file:

    content = file.read()

    print(content)
```

Writing Multiple Lines

```
with open("students.txt", "w") as file:

    file.write("John\n")
    file.write("Alice\n")
    file.write("David\n")
```

File Content

```
John
Alice
David
```

writelines()

Writes multiple lines.

```
students = [
    "John\n",
    "Alice\n",
```

```
"David\n"  
]  
  
with open("students.txt","w") as file:  
  
    file.writelines(students)
```

File Pointer

The file pointer indicates the current position.

Example:

```
file = open("student.txt","r")  
  
print(file.tell())  
  
file.read(5)  
  
print(file.tell())  
  
file.close()
```

Output

```
0  
5
```

seek()

Moves pointer position.

```
file = open("student.txt","r")  
  
file.seek(0)  
  
print(file.read())  
  
file.close()
```

Working with Binary Files

Write Binary

```
with open("data.bin","wb") as file:  
  
    file.write(b"Python")
```

Read Binary

```
with open("data.bin","rb") as file:  
    print(file.read())
```

Output

```
b'Python'
```

Checking File Existence

```
Using os module.  
import os  
  
if os.path.exists("student.txt"):  
    print("File Exists")  
  
else:  
    print("File Not Found")
```

Deleting a File

```
import os  
  
os.remove("student.txt")
```

Creating Directories

```
import os  
  
os.mkdir("PythonFiles")
```

Removing Directory

```
import os  
  
os.rmdir("PythonFiles")
```

Exception Handling in File Operations

Example

```
try:

    file = open("abc.txt","r")

    print(file.read())

except FileNotFoundError:

    print("File Not Found")
```

Output

```
File Not Found
```

CSV File Handling

CSV = Comma Separated Values

Writing CSV

```
import csv

with open("students.csv","w",newline="") as file:

    writer = csv.writer(file)

    writer.writerow(["Name","Marks"])

    writer.writerow(["John",85])
```

Reading CSV

```
import csv

with open("students.csv","r") as file:

    reader = csv.reader(file)

    for row in reader:

        print(row)
```

Output

```
['Name', 'Marks']
['John', '85']
```

JSON File Handling

Writing JSON

```
import json

student = {
    "name": "John",
    "marks": 85
}

with open("student.json", "w") as file:
    json.dump(student, file)
```

Reading JSON

```
import json

with open("student.json", "r") as file:
    data = json.load(file)

    print(data)
```

Output

```
{'name': 'John', 'marks': 85}
```

Practical Example 1: Student Record System

```
with open("students.txt", "a") as file:
    name = input("Enter Name: ")
    file.write(name + "\n")
```

Practical Example 2: Log File Analyzer

```
with open("security.log", "r") as file:
    for line in file:
        if "Failed" in line:
            print(line)
```

Practical Example 3: Employee Database

```
employee = {  
    "id":101,  
    "name":"John",  
    "salary":50000  
}  
  
import json  
  
with open("employee.json","w") as file:  
    json.dump(employee,file)
```

Applications of File Handling

Data Science

dataset.csv

Used for:

- Reading datasets
- Data preprocessing

Machine Learning

training_data.csv

Used for:

- Model training
- Data storage

Web Development

config.json

Used for:

- Configuration files

Cyber Security

security.log

Used for:

- Log monitoring
- Threat detection

Digital Forensics

```
evidence.txt
```

Used for:

- Evidence collection
- Log preservation

Common File Handling Errors

FileNotFoundError

```
open("abc.txt", "r")
```

Output:

```
FileNotFoundError
```

PermissionError

Occurs when access is denied.

```
PermissionError
```

IsADirectoryError

Occurs when attempting to open a folder as a file.

```
IsADirectoryError
```

Advantages of File Handling

1. Permanent data storage.
2. Easy data retrieval.
3. Supports large datasets.
4. Enables data sharing.

5. Essential for real-world applications.

Best Practices

1. Always use `with open()` whenever possible.
2. Close files properly.
3. Handle exceptions.
4. Use appropriate file modes.
5. Validate file existence before access.
6. Store sensitive data securely.

Summary

Python File Handling is the process of creating, opening, reading, writing, appending, and managing files using Python programs. Python provides the `open()` function and various file modes (`r`, `w`, `a`, `x`, etc.) to perform file operations efficiently. File handling is a fundamental concept used in software development, data science, machine learning, web applications, cyber security, digital forensics, database systems, and enterprise applications for storing and managing persistent data.